

API PLATFORM CONFERENCE

API SECURED, THE MODERN WAY

Des APIs **sécurisées** sans perdre la tête

OAuth2

OIDC

Rate limiting

PKCE

JWT

SSO



Mathieu Santostefano

Salut !

Mathieu Santostefano

Tech Expert **Sensio**Labs

 **Symfony** Core Team Member

 **Clever Cloud** Ambassador

 [@welcomattic.com](https://twitter.com/welcomattic)

 [@welcomattic](https://github.com/welcomattic)



API PLATFORM
CONFERENCE

Une API "Sécurisée" ?



Autorisation

OAuth2

Que puis-je faire ?

Déléguer l'accès : scopes, permissions



Authentification

OIDC

Qui êtes-vous ?

Alice prouve qui elle est,
l'app reçoit des tokens



Rate limiting

Et bien plus...

01

Autorisation

OAuth2

Déléguer l'accès à ses ressources,
sans partager ses identifiants.



Focus du jour

Le scénario au nom de l'utilisateur
(Authorization Code Flow).

*Sans utilisateur dans la boucle, une app qui appelle une API pour son propre compte :
Client Credentials Flow, hors focus.*

Alice, PhotoPrint et CloudPics

LE DÉCOR

 Alice héberge ses photos sur **CloudPics**.

 Depuis **PhotoPrint**, une app web tierce, elle veut **faire imprimer ses photos CloudPics**.

LA CONTRAINTE

 Sans jamais lui demander son **mot de passe CloudPics**.

LE FLOW

 PhotoPrint **redirige** Alice vers CloudPics, qui lui demande son accord.

Autorises-tu PhotoPrint à accéder à tes photos CloudPics ?

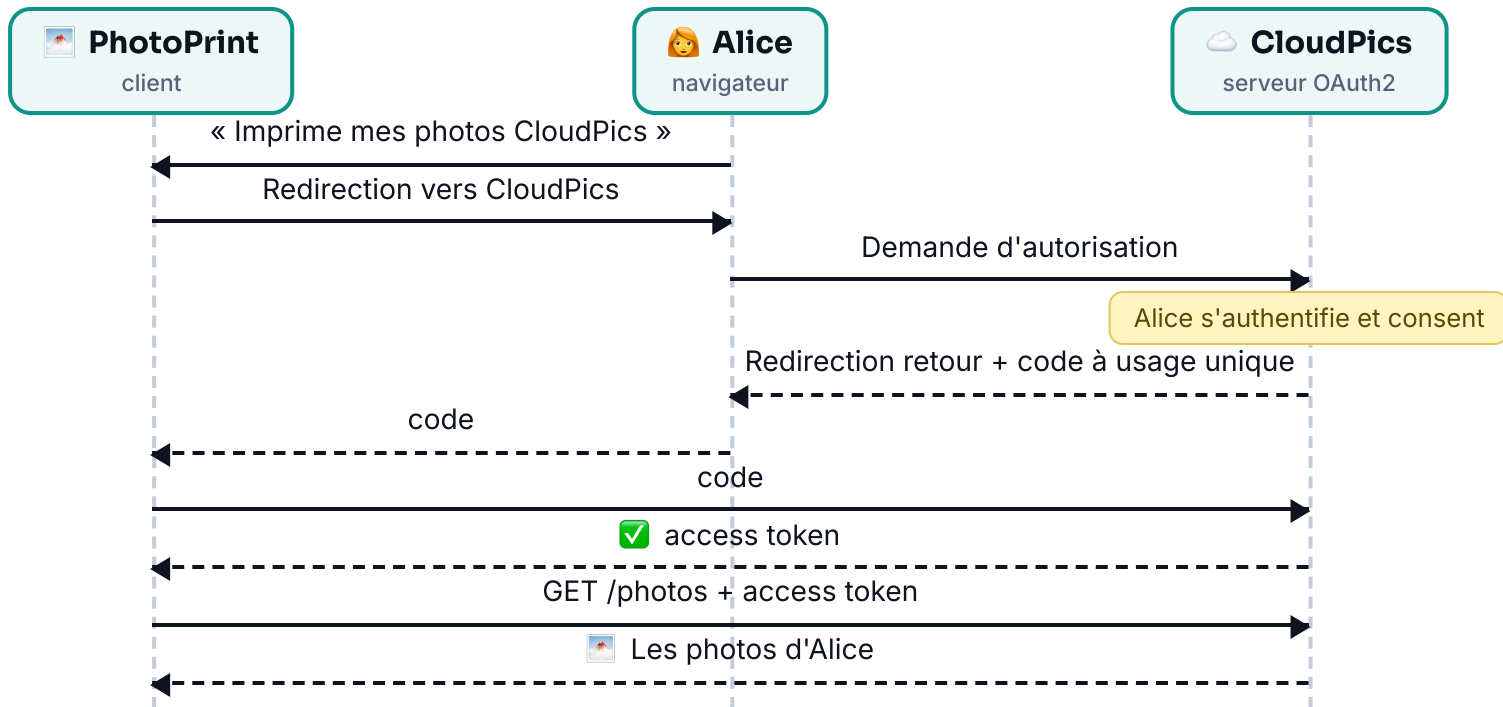
 Alice **autorise** : CloudPics émet un **code** à usage unique.

 PhotoPrint **échange** ce code contre un **access token**

 PhotoPrint **accède** aux photos d'Alice grâce à l'access token

● CloudPics joue le rôle de serveur OAuth2, et **PhotoPrint** d'application cliente

Le flow, étape par étape



Les parties prenantes



Alice

Resource owner

Elle seule peut accorder l'accès à ses photos.



PhotoPrint

Client public

Une SPA qui tourne chez Alice, sans secret possible.
Demande les photos **pour le compte** d'Alice.



CloudPics

Authorization server

+ Resource server
Émet les access tokens, et héberge les photos.

Un client **public** tourne chez l'utilisateur (SPA, mobile) et ne peut garder aucun secret. Un client **confidentiel** tourne sur son propre serveur et s'authentifie avec un `client_secret`.

La première faiblesse

Le code d'autorisation transite par le navigateur, dans l'URL de redirection

 **Web** : code dans l'**URL de redirection** → historique, logs, `Referer`

 **Mobile** : une app malveillante prend le **même URL scheme** (`monapp://`) → capte le code

**Le code d'autorisation peut être volé.
Et un client public n'a aucun secret pour compenser.**

La parade : PKCE

Proof Key for Code Exchange



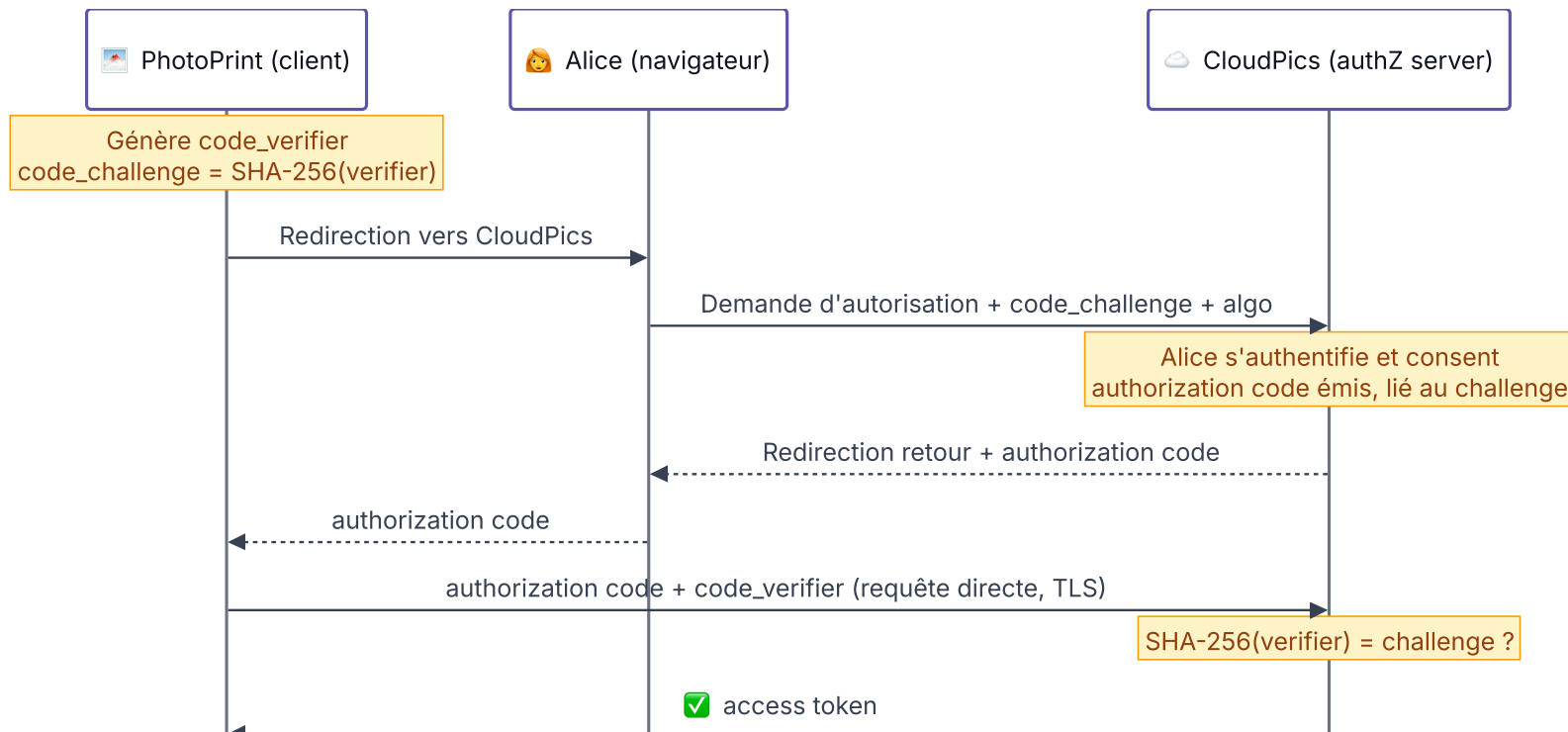
code_verifier : un secret aléatoire, connu du seul client



code_challenge : son empreinte SHA-256, envoyée en premier

Le challenge est irréversible : sans le verifier, un code volé ne vaut rien.

PKCE en séquence



- OAuth 2.1 (toujours un draft IETF) : PKCE obligatoire pour **tous** les clients.

Le token ne prouve rien : le détenir suffit

```
GET /api/photos HTTP/1.1
Host: api.cloudpics.example
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9 ...
```

 **Bearer** = « au porteur » : aucune preuve n'est demandée au client

 Volé, il est **indiscernable** d'un token légitime. L'API ne peut pas trancher

 **TLS obligatoire** : sur le réseau, lire le token, c'est pouvoir l'utiliser

 **Jamais dans l'URL**, même en HTTPS : historique, logs serveur et `Referer` la conservent en clair

Par défaut, rien n'empêche l'usage d'un token volé.
On ne corrige pas ça, on limite sa durée de validité.

Votre API peut être son propre serveur OAuth2



league/oauth2-server-bundle



Votre API = **authorization server** + **resource server**



Émission + validation des tokens, contrôle des **scopes**

Possible. Pas ce que je recommande.

Le bundle ne fait pas tout à votre place

 **Login d'Alice** : `/authorize` exige un utilisateur connecté, le formulaire de login reste à coder

 **Consentement** : le bundle émet un événement, l'écran « Autorises-tu PhotoPrint ? » reste à coder

 **Rotation** des clés de signature

 **MFA**, mot de passe oublié, révocation de sessions

 **Audit** : qui a autorisé quoi, et quand

Vous maintenez un serveur d'autorisation

C'est un produit à part entière.

Trop lié à votre API : l'un peut faire tomber l'autre

Déléguer le serveur, pas la sécurité

Des acteurs dont c'est le **seul métier** émettent les tokens.
Votre API se contente de les **vérifier**.



OAuth2 seul : données utilisateur **non standardisées**.
OIDC ajoute l'**ID token** : l'identité dans un **format cohérent**.

02

Authentication

OIDC ajoute une couche d'identité à OAuth2

 **ID Token** : JWT avec les claims d'identité

 **Claims standards** : sub, aud, iss, exp...

 **Discovery** : endpoint `.well-known`

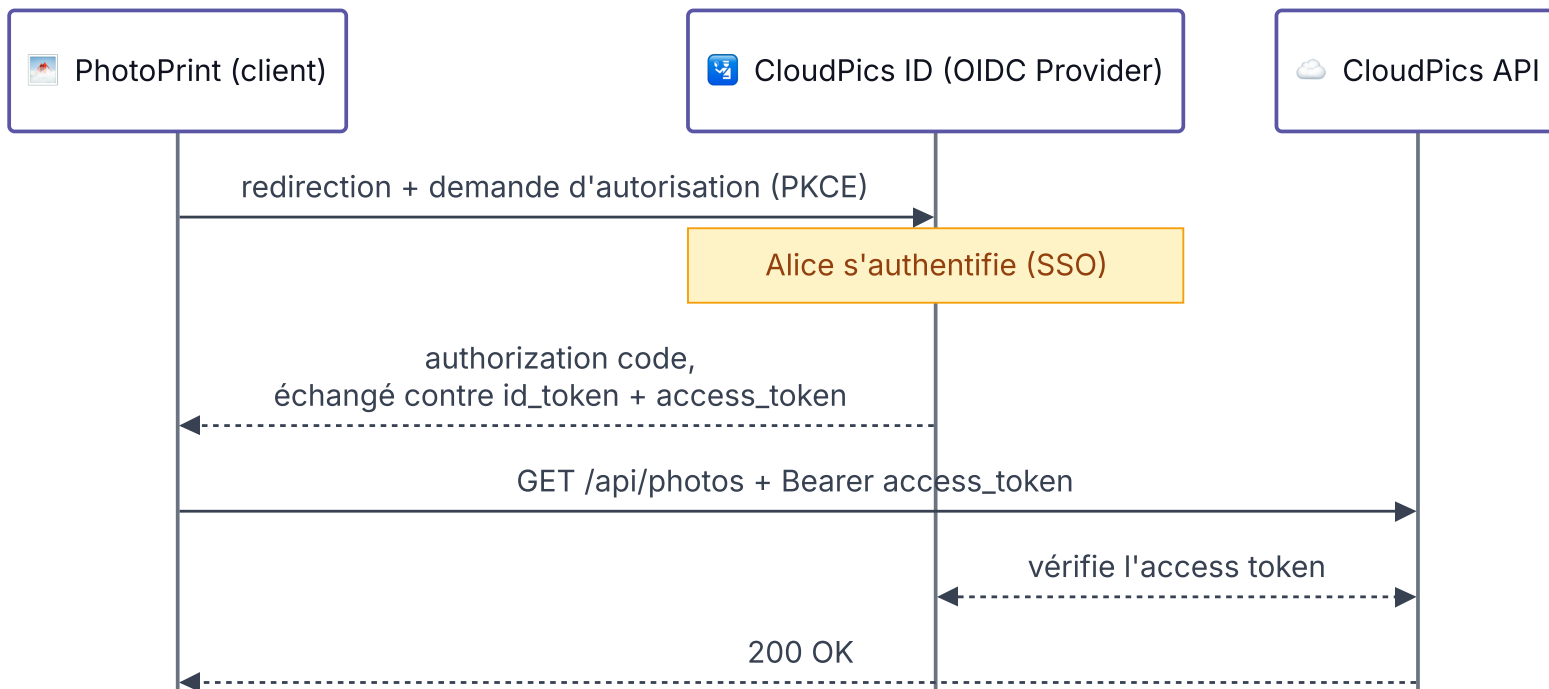
 **UserInfo** : données utilisateur supplémentaires

OIDC Provider

Un **Provider dédié** émet les tokens et fournit les identités.
Indépendamment de votre API.

*Dans notre histoire : CloudPics confie l'identité à **CloudPics ID**. Son API ne garde que les photos.*

Le client obtient les tokens, l'API les vérifie



Les parties prenantes



Alice

End-User

Utilisateur humain



PhotoPrint

Relying Party (RP)

Le client qui réclame
l'authentification et les claims.



CloudPics ID

OpenID Provider (OP)

Authentifie Alice, puis fournit
les claims au RP.



Et **CloudPics API**, votre API Platform ?

Un **Resource Server** : une dénomination OAuth2, que la spec OIDC ne change pas.

Deux tokens, deux destinataires



ID token

Pour l'**application cliente**.

Qui est l'utilisateur, et comment il s'est authentifié.

> Carte d'identité



Access token

Pour l'**API**.

Ce que le porteur a le droit de faire.

> Billet de concert



L'ID token n'est **pas** une clé d'accès à l'API. Seul l'**access token** l'est.

Nous n'implémentons pas notre propre OIDC Provider.

C'est un métier à part entière.

Open source ou SaaS : tous parlent OIDC

OPEN SOURCE



Authelia



Authentik



FerrisKey



Ory Hydra



PocketID



Rauthy



Casdoor



Dex



Keycloak

LES DEUX



Gravitee AM



Zitadel



SuperTokens

SAAS



Cidaas



Cloud-IAM



Please Open
It



Auth0



AWS
Cognito



Clerk



Microsoft
Entra ID



Firebase
Auth

Kinde

Kinde



LoginRadius



Okta



PingOne

 origine européenne. Au milieu, ceux qui existent **dans les deux modèles**.

Besoin de SSO social ?

Quelques client credentials à configurer, et c'est branché.



Google



Microsoft



PayPal



Facebook



GitHub



GitLab



LinkedIn



Bitbucket

Pas une ligne de code, **que de la configuration**. Keycloak fournit douze connecteurs en standard.

Votre API redevient un simple resource server

👉 Les **apps clientes** redirigent les utilisateurs vers l'**OIDC Provider**, qui les authentifie et **émet les tokens**

👤 Les **comptes** vivent dans l'**OIDC Provider**, plus dans votre base

🧩 Votre **API** ne fait que **vérifier** les tokens

🏦 **Aucun** écran de login ni de consentement à coder : elle se concentre sur le **métier**

Symfony vérifie les access tokens nativement

- 🔌 Authenticator `access_token` dans le firewall
- 📧 Lit l'en-tête `Authorization: Bearer` par défaut
- 🧩 Un **token handler** décide *comment* valider
- 🎯 Trois handlers natifs : `oidc`, `oidc_user_info` et `oauth2`

Trois façons de vérifier un token



`oidc`

Offline

Vérifie la **signature** du JWT avec les clés publiques du Provider, puis ses claims (`exp` , `aud` , `iss`).



`oidc_user_info`

Online

Présente le token à l'endpoint **userinfo** du Provider, qui le valide et renvoie les claims de l'utilisateur.



`oauth2`

Online

Présente le token à l'endpoint d'**introspection** du serveur d'autorisation, qui répond `active` et les claims.

Le handler change, pas le reste : même firewall, mêmes rôles, même `is_granted` .

Offline ou online : un arbitrage, pas un gagnant

	oidc (offline)	oidc_user_info (online)	oauth2 (online)
Appel réseau	Aucun	Un par requête	Un par requête
Révocation d'un token	Visible à l'expiration	Immédiate	Immédiate
Provider indisponible	L'API continue de servir	L'API ne répond plus	L'API ne répond plus
Validation de aud	Oui	Aucune	Oui

Access tokens courts + offline : le meilleur compromis dans la majorité des cas.

Dans la démo, tout finit dans security

```
// src/Entity/Photo.php
#[ApiResponse(security: "is_granted('ROLE_PHOTOS_READ')")]
#[GetCollection]
#[Get(security: "is_granted('ROLE_PHOTOS_READ') and object.owner == user.getUserIdentifier()")]
#[Post(security: "is_granted('ROLE_PHOTOS_WRITE')")]
class Photo { /* ... */ }
```

🗨️ Rôles du Provider $\neq$ rôles Symfony : `OidcUser` n'arrive qu'avec `ROLE_USER`

🔄 Les mapper vers `ROLE_PHOTOS_*` : un **UserProvider** écrit à la main, **obligatoire**

🏠 `object.owner == user` : la règle métier, connue de l'API seule

📄 Symfony 8.2 : `OAUTH2_SCOPE(photos:read)` lit le scope, sans mapping

**Un access token dit ce que son porteur
peut faire. L'API l'applique ici, en déclaratif.**

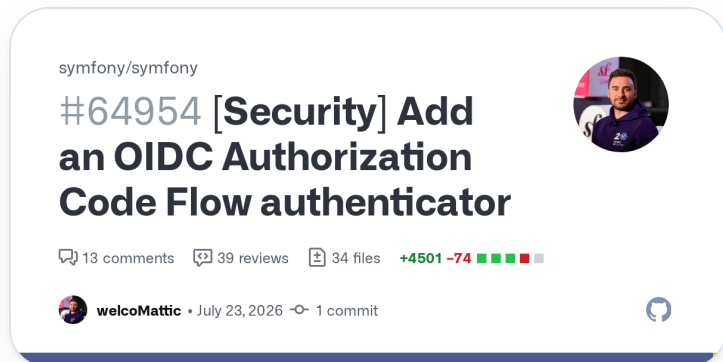
Et si le client est lui aussi une app Symfony ?

PhotoBook, un autre service tiers, client **confidentiel** : il tourne sur son serveur et garde un `client_secret`.

✓ **Vérifier** un access token : natif (`access_token`)

🎉 **Initier** le flow : l'authenticator `oidc_login`, natif dans Symfony 8.2

🛣️ **PKCE**, échange du code, **signature de l'ID token** vérifiée, logout, refresh



Mergé le 2 septembre, une dizaine de PR de suite depuis. Livré en novembre 2026.

Pas de bundle dans la démo : PhotoBook tourne déjà dessus.

03

Rate Limiting

Symfony Rate Limiter : une simple config

🎯 Contre les **abus** : brute force, scraping, clients trop gourmands

🔧 Une limite, un intervalle, un stockage. Un limiter **par usage**

```
# config/packages/rate_limiter.yaml
framework:
  rate_limiter:
    api:
      policy: 'token_bucket'
      limit: 100
      rate: { interval: '1 second', amount: 10 }
```

En prod, N instances = N compteurs : un pool de cache **partagé** (Redis) via `cache_pool` .

La limite se déclare sur l'opération

🧩 Un **state provider** décoré : la limite s'applique **par opération**

🛡️ `TooManyRequestsHttpException` → **HTTP 429** automatique

🔑 Clé du quota : le `sub` du token, pas l'IP

```
// src/Entity/Photo.php
#[ApiResponse(security: "is_granted('ROLE_PHOTOS_READ')")]
#[GetCollection(provider: RateLimitedProvider::class)]
#[Post(security: "is_granted('ROLE_PHOTOS_WRITE')")]
class Photo
{
    // ...
}
```

Une opération limitée, l'autre non.

Le même style déclaratif que `security:`

Le provider consomme un jeton, puis délègue

```
// src/State/RateLimitedProvider.php
final class RateLimitedProvider implements ProviderInterface
{
    public function __construct(
        #[Autowire(service: 'api_platform.doctrine.orm.state.collection_provider')]
        private ProviderInterface $inner,
        private RateLimiterFactoryInterface $apiLimiter,
        private Security $security,
    ) {}

    public function provide(Operation $op, array $uriVariables = [], array $context = []): object|array|null {
        $key = $this->security->getUser()?->getUserIdentifier(); // le sub du token
        $limit = $this->apiLimiter->create($key)->consume();
        if (!$limit->isAccepted()) {
            throw new TooManyRequestsHttpException($limit->getRetryAfter()->getTimestamp() - time());
        }

        return $this->inner->provide($op, $uriVariables, $context);
    }
}
```



L'API Gateway

La surprise !

Est-ce vraiment le job de votre API ?

🔑 Auth, autorisation, rate limiting, protection...

⌚ Un travail à plein temps

🔔 API Gateway / API Management : c'est leur métier

Choisissez votre préférée !

Open source ou commerciale, auto-hébergée ou SaaS, le choix est vaste !

ORIGINE EUROPÉENNE



Gravitee



KrakenD



Otoroshi



Traefik

RESTE DU MONDE



Amazon API Gateway



Apigee



Apisix



Kong

Délégez !

Trois domaines entiers que votre API n'a plus à porter.



Accès

Vérification des tokens
Autorisation grossière :
scopes, routes
CORS (préflight + en-têtes)



Protection

WAF : SQLi, XSS, OWASP
Détection de bots
IP allow/deny, géo-blocage



Exploitation

Rate limiting
Logging
Monitoring

**Tout ça en amont de votre API
et sans "polluer" votre code applicatif.**

L'API garde `is_granted` et le contrôle du propriétaire, et revérifie le JWT : en offline, ça ne coûte rien.

À retenir

1

Les standards existent : utilisez-les

JWT, OAuth 2.0, OIDC. **N'inventez pas le vôtre.**

2

Les outils modernes simplifient tout

Symfony Security, League, l'authenticator `access_token`, les OIDC Providers. **Le gros du travail est déjà fait.**

3

La sécurité est une fonctionnalité

Pas une option, pas un « nice to have ». **Dès le premier jour.**

4

Équipez vos APIs

API Gateway / API Management. **Déléguez ce qui peut l'être.**

Ressources



Spécifications

[RFC 6749](#) OAuth 2.0

[RFC 6750](#) Bearer Token

[RFC 7636](#) PKCE

[RFC 7519](#) JWT

[RFC 9068](#) JWT access token

[RFC 9700](#) bonnes pratiques

[RFC 9449](#) DPoP

[RFC 7662](#) Introspection

[OpenID Connect](#) la spec OIDC



Documentation

[Symfony Access Token](#)

[PR 64954](#) l'authenticator

oidc_login (8.2)

[API Platform Security](#)

[Symfony Rate Limiter](#)

[League OAuth2 Server Bundle](#)

[drenso/symfony-oidc](#) en

attente



Outils et démo

[Le dépôt de ce talk](#) deck et
démo

[jwt.io](#) debugger JWT

[OAuth 2.0 Playground](#)

[OWASP API Security Top 10](#)

[RS256 vs HS256](#)

Allez voir Florent demain midi !

Les en-têtes HTTP comme première ligne de défense des APIs et du front

Florent Morselli Salle 1 11h50 - 12h10





SÉCURITÉ **BONNES PRATIQUES**

Les en-têtes HTTP comme première ligne de défense des APIs et du front 🇫🇷

JOUR 2 - SALLE 1

18 sept. 2026 · 11:50 AM - 12:10 PM

par Florent Morselli

Il couvre d'autres aspects de sécurité avec plein d'autres acronymes rigolos : CSP, CORS, COOP, COEP.

La démo est à vous

github.com/welcoMattic/api-secured-the-modern-way

```
cd demo

castor start          # les 4 acteurs, en local
cd infra && tofu apply # les mêmes, sur Clever Cloud
```

**Un client public, un client
confidentiel, le même Provider.**

L'apply crée l'add-on Keycloak, les trois apps et leurs variables, puis déploie le code. Clever attribue les domaines, il n'y a rien à réserver. Seul l'import du realm reste un script.



API SECURED, THE MODERN WAY

Merci !

Des questions ?



@welcomattic.com



welcoMatic



mathieu.santostefano@sensiolabs.com

Annexe : les JW* expliqués

Acronyme	Nom complet	Rôle	Métaphore
JWT	JSON Web Token	La structure des claims, toujours emballée en JWS ou JWE	Le courrier
JWS	JSON Web Signature	Intégrité et authenticité : un token signé	Une enveloppe transparente scellée
JWE	JSON Web Encryption	Confidentialité : un token chiffré	Une boîte opaque et verrouillée
JWKS	JSON Web Key Set	Les clés publiques du Provider, pour vérifier les signatures	Le trousseau